

JavaScript Lernzettel - Grundlagen

Für Cedric - einfach erklärt mit Bedeutung, Mini-Code und Merksatz

Ziel: Code nicht nur kopieren, sondern verstehen: Was wird gesucht? Was wird gespeichert? Was wird geändert? Was wird ausgeführt?

1. Grundidee

JavaScript macht Webseiten interaktiv. HTML erstellt die Elemente, CSS gestaltet sie, JavaScript sucht Elemente, reagiert auf Aktionen und veraendert Inhalte.

HTML: `<button id="start-btn">Klick mich</button>`
JavaScript: `const button = document.getElementById("start-btn");`

Merksatz: HTML baut das Element. JavaScript findet es ueber die ID und kann damit arbeiten.

2. Die wichtigsten Basics

Begriff	Bedeutung	Mini-Code	Merksatz
const	Erstellt eine Variable, die nicht neu zugewiesen werden soll.	<code>const name = "Cedric";</code>	const = fester Speichername.
let	Erstellt eine Variable, die spaeter veraendert werden darf.	<code>let total = 0; total = total + 25;</code>	let = Wert darf sich aendern.
Variable	Ein Speicherplatz mit Namen.	<code>const username = "Cedric";</code>	Variable = etwas unter einem Namen speichern.
document.getElementById()	Sucht ein HTML-Element ueber seine ID.	<code>const button = document.getElementById("start-btn");</code>	Suche das Element mit der ID start-btn.
.value	Holt den Inhalt aus einem Eingabefeld.	<code>const name = nameInput.value;</code>	value = Was steht im Feld?
.trim()	Entfernt Leerzeichen vorne und hinten.	<code>const name = nameInput.value.trim();</code>	trim = Leerzeichen am Rand entfernen.
.textContent	Aendert den sichtbaren Text eines Elements.	<code>output.textContent = "Hallo " + name;</code>	textContent = Text auf Webseite aendern.
console.log()	Zeigt etwas in der Entwicklerkonsole an.	<code>console.log(name);</code>	Zum Pruefen, nicht fuer die Webseite.

3. Aktionen und Funktionen

Begriff	Bedeutung	Mini-Code	Merksatz
addEventListener	Wartet auf eine Aktion, z. B. Klick oder Tastendruck.	<pre>button.addEventListener("click", function () { console.log("geklickt"); });</pre>	Warte auf Klick, dann führe Code aus.
"click"	Das Ereignis: Der Nutzer klickt auf ein Element.	<pre>button.addEventListener("click", ...);</pre>	click = Klick-Aktion.
function	Eine gespeicherte Aufgabe / ein Codeblock.	<pre>function begruessen() { console.log("Hallo"); }</pre>	function = Aufgabe speichern.
funktionName()	Führt eine Funktion sofort aus.	<pre>begruessen();</pre>	Mit Klammern = jetzt ausführen.
funktionName ohne ()	Übergibt die Funktion für später, z. B. an einen EventListener.	<pre>button.addEventListener("click", begruessen);</pre>	Ohne Klammern = später ausführen.
if	Prüft, ob eine Bedingung stimmt.	<pre>if (name === "") { output.textContent = "Bitte Namen eingeben"; }</pre>	if = wenn Bedingung stimmt.
else	Läuft, wenn die if-Bedingung nicht stimmt.	<pre>} else { output.textContent = "Hallo " + name; }</pre>	else = sonst / andernfalls.
===	Vergleicht: ist genau gleich.	<pre>if (name === "")</pre>	=== = genau gleich.
""	Leerer Text. Nichts zwischen den Anführungszeichen.	<pre>if (name === "")</pre>	"" = leer.
 	Oder. Eine von mehreren Bedingungen reicht.	<pre>if (name === "" amount <= 0)</pre>	= oder.
return	Stoppt die Funktion an dieser Stelle.	<pre>if (name === "") { return; }</pre>	return = hier abbrechen.

4. Elemente erstellen und anzeigen

Begriff	Bedeutung	Mini-Code	Merksatz
createElement	Erstellt ein neues HTML-Element mit JavaScript.	<pre>const listItem = document.createElement("li");</pre>	Erstelle neuen Listenpunkt.
className	Gibt einem Element eine CSS-Klasse per JavaScript.	<pre>listItem.className = "expense-item";</pre>	className = Name fuer CSS setzen.
innerHTML	Schreibt HTML-Inhalt in ein Element.	<pre>listItem.innerHTML = `<span>\${expense.name}</span>`;</pre>	innerHTML = HTML in Element reinpacken.
\${variable}	Setzt den Wert einer Variable in einen Template-String ein.	<pre>`Hallo \${name}`</pre>	Setze den Wert hier ein.
appendChild	Haengt ein Element in ein anderes Element rein.	<pre>expenseList.appendChild(listItem);</pre>	appendChild = auf Webseite einfüegen.
remove()	Entfernt ein Element von der Webseite.	<pre>listItem.remove();</pre>	remove = Element loeschen.
querySelector	Sucht ein Element mit CSS-Schreibweise.	<pre>const deleteBtn = listItem.querySelector(".delete-btn");</pre>	Punkt . = Klasse, Raute # = ID.

Mini-Ablauf: Listeneintrag anzeigen

```
const listItem = document.createElement("li");
listItem.className = "expense-item";
listItem.innerHTML = `<span>${expense.name}</span><span>${expense.amount} Euro</span>`;
expenseList.appendChild(listItem);
```

Bedeutung: Neuen Listenpunkt erstellen, CSS-Klasse geben, Inhalt reinpacken, in die Liste einfüegen.

5. Listen, Arrays und Wiederholung

Begriff	Bedeutung	Mini-Code	Merksatz
Array []	Eine Liste mit mehreren Werten.	<pre>let expenses = [];</pre>	[] = leere Liste.
push	Fuegt etwas zu einer Liste hinzu.	<pre>expenses.push({ name: expenseName, amount: expenseAmount });</pre>	push = zur Liste hinzufuegen.
Objekt {}	Speichert zusammengehoerige Daten.	<pre>{ name: "Einkaufen", amount: 25 }</pre>	Objekt = Name + Betrag zusammen.
forEach	Geht jeden Eintrag einer Liste einzeln durch.	<pre>expenses.forEach(function (expense) { console.log(expense.name); });</pre>	forEach = jeden Eintrag durchgehen.
renderExpenses	Eigene Funktion, die alle Ausgaben neu auf der Webseite anzeigt.	<pre>function renderExpenses() { expenseList.innerHTML = ""; expenses.forEach(...); }</pre>	render = Anzeige neu aufbauen.

6. localStorage - Speichern im Browser

Begriff	Bedeutung	Mini-Code	Merksatz
localStorage	Lokaler Speicher im Browser. Bleibt nach Refresh erhalten.	<pre>localStorage.setItem(...);</pre>	Browser-Speicher.
setItem	Speichert etwas im localStorage.	<pre>localStorage.setItem("expenses", JSON.stringify(expenses));</pre>	setItem = speichern.
getItem	Holt etwas aus dem localStorage.	<pre>localStorage.getItem("expenses");</pre>	getItem = holen/laden.
JSON.stringify	Packt Daten so ein, dass localStorage sie speichern kann.	<pre>JSON.stringify(expenses)</pre>	stringify = einpacken.
JSON.parse	Packt gespeicherte Daten wieder aus, damit JavaScript sie nutzen kann.	<pre>JSON.parse(localStorage.getItem("expenses"))</pre>	parse = auspacken.
 []	Falls nichts gespeichert ist, nimm eine leere Liste.	<pre>let expenses = JSON.parse(localStorage.getItem("expenses")) [];</pre>	Wenn nichts da ist: leere Liste.

7. Wichtige Mini-Beispiele zum Ueben

Eingabefeld auslesen

```
const input = document.getElementById("username");
const username = input.value;
console.log(username);
```

Sucht das Eingabefeld, holt den Inhalt und zeigt ihn in der Konsole.

Text auf Webseite aendern

```
const output = document.getElementById("output");
output.textContent = "Hallo Cedric";
```

Sucht den Ausgabebereich und aendert seinen Text.

Button-Klick

```
const button = document.getElementById("start-btn");
button.addEventListener("click", function () {
    console.log("Button geklickt");
});
```

Wartet auf Klick und fuehrt dann den Code aus.

if / else

```
if (username === "") {
    output.textContent = "Bitte Namen eingeben";
} else {
    output.textContent = "Hallo " + username;
}
```

Wenn leer: Fehlermeldung. Sonst: Begrueessung.

Speichern

```
localStorage.setItem("expenses", JSON.stringify(expenses));
```

Speichert die Ausgabenliste im Browser.

Laden

```
let expenses = JSON.parse(localStorage.getItem("expenses")) || [];
```

Laedt die gespeicherte Liste oder nimmt eine leere Liste.

8. Die wichtigsten Merksaetze

HTML erstellt Elemente.
CSS gestaltet Elemente.
JavaScript sucht, veraendert und steuert Elemente.

id = eindeutiger Name fuer JavaScript.
class = Name fuer CSS.
.value = Inhalt aus Eingabefeld holen.
textContent = sichtbaren Text aendern.
addEventListener = auf Aktion warten.
if = wenn Bedingung stimmt.
else = sonst.
function = gespeicherte Aufgabe.
() = Funktion ausfuehren.
Array = Liste.
push = zur Liste hinzufuegen.
forEach = jeden Listeneintrag durchgehen.
localStorage = Speicher im Browser.
setItem = speichern.
getItem = laden.
JSON.stringify = einpacken.
JSON.parse = auspacken.